

Week 7 Assignment Report

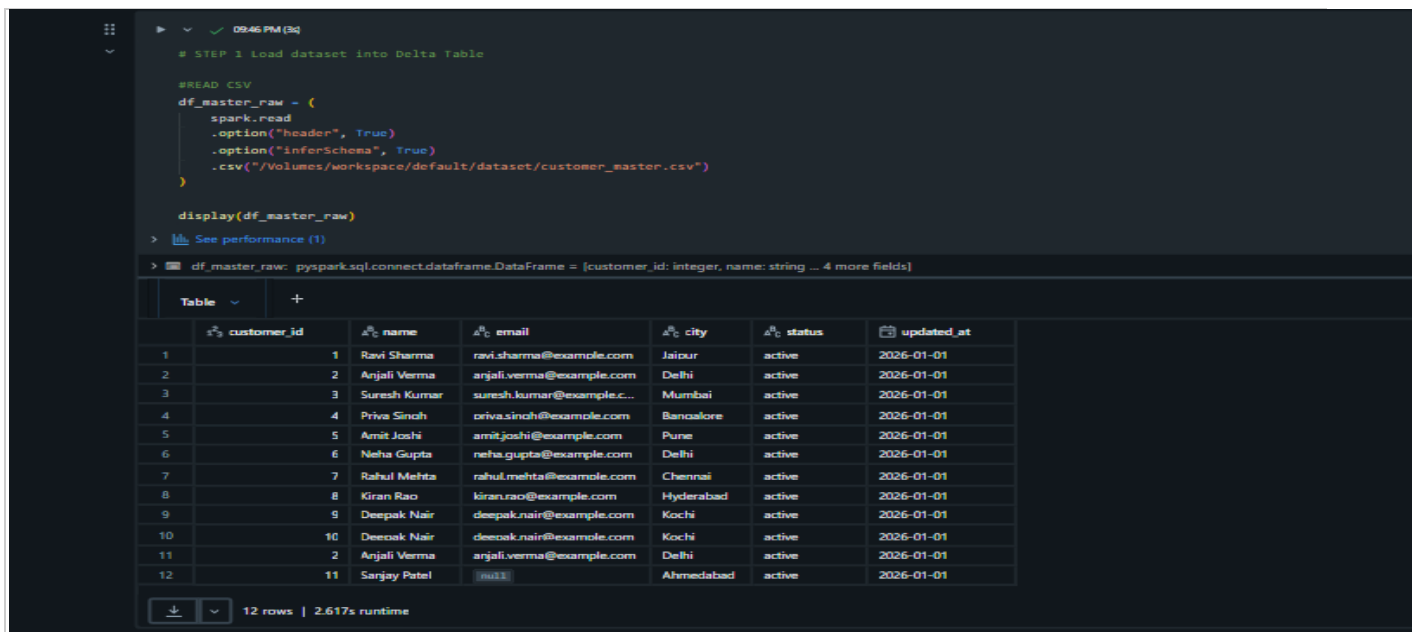
Delta Lake — Incremental Load & Slowly Changing Dimension (SCD) using MERGE

Objective

In this task, a small Slowly Changing Dimension (SCD) style workflow was built using Delta Lake — customer master data was loaded into a Delta table, cleaned, and then incremental (new/updated) records were applied to the target table using a MERGE operation, with the final result validated at the end.

Step 1 — Loading the CSV and Creating a Delta Table

First, the customer_master.csv file was read using Spark. This raw dataframe was then saved as a Delta table, so that advanced operations like MERGE could be performed on it later.



The screenshot shows a Databricks notebook interface. At the top, the time is 09:46 PM (3q). The code cell contains the following Python code:

```
# STEP 1 load dataset into Delta Table

#READ CSV
df_master_raw = (
    spark.read
    .option("header", True)
    .option("inferSchema", True)
    .csv("/Volumes/workspace/default/dataset/customer_master.csv")
)

display(df_master_raw)
```

Below the code, there is a link to "See performance (1)". The output of the code is a DataFrame with the following schema: [customer_id: integer, name: string ... 4 more fields]. The DataFrame is displayed as a table with 12 rows and 7 columns: customer_id, name, email, city, status, and updated_at. The table shows 12 records of customer data, including names like Ravi Sharma, Anjali Verma, Suresh Kumar, Priva Singh, Amit Joshi, Neha Gupta, Rahul Mehta, Kiran Rao, Deepak Nair, Deepak Nair, Anjali Verma, and Sarjay Patel. The email addresses are all from example.com, and the status is "active" for all. The updated_at date is 2026-01-01 for all records. The table is titled "Table" and has a download button. The bottom of the table shows "12 rows | 2.617s runtime".

	customer_id	name	email	city	status	updated_at
1	1	Ravi Sharma	ravi.sharma@example.com	Jaiour	active	2026-01-01
2	2	Anjali Verma	anjali.verma@example.com	Delhi	active	2026-01-01
3	3	Suresh Kumar	suresh.kumar@example.c...	Mumbai	active	2026-01-01
4	4	Priva Singh	priva.singh@example.com	Banalore	active	2026-01-01
5	5	Amit Joshi	amitjoshi@example.com	Pune	active	2026-01-01
6	6	Neha Gupta	neha.gupta@example.com	Delhi	active	2026-01-01
7	7	Rahul Mehta	rahul.mehta@example.com	Chennai	active	2026-01-01
8	8	Kiran Rao	kiran.rao@example.com	Hyderabad	active	2026-01-01
9	9	Deepak Nair	deepak.nair@example.com	Kochi	active	2026-01-01
10	10	Deepak Nair	deepak.nair@example.com	Kochi	active	2026-01-01
11	2	Anjali Verma	anjali.verma@example.com	Delhi	active	2026-01-01
12	11	Sarjay Patel	[null]	Ahmedabad	active	2026-01-01

Output - customer_master.csv read into a raw dataframe

The screenshot shows a Databricks notebook with two cells. The first cell contains code to save a DataFrame as a Delta Table. The second cell contains code to display the Delta Table as a Spark table.

```

# Save as Delta Table
df_master_raw.write
  .format("delta")
  .mode("overwrite")
  .saveAsTable("delta_table")

display(spark.table("delta_table"))

```

The output of the second cell is a table with 12 rows and 7 columns. The columns are: customer_id, name, email, city, status, and updated_at. The data is as follows:

	customer_id	name	email	city	status	updated_at
1	1	Ravi Sharma	ravi.sharma@example.com	Jaipur	active	2026-01-01
2	2	Anjali Verma	anjali.verma@example.com	Delhi	active	2026-01-01
3	3	Suresh Kumar	suresh.kumar@example.c...	Mumbai	active	2026-01-01
4	4	Priva Singh	priva.singh@example.com	Bangalore	active	2026-01-01
5	5	Amit Joshi	amit.joshi@example.com	Pune	active	2026-01-01
6	6	Neha Gupta	neha.gupta@example.com	Delhi	active	2026-01-01
7	7	Rahul Mehta	rahul.mehta@example.com	Chennai	active	2026-01-01
8	8	Kiran Rao	kiran.rao@example.com	Hyderabad	active	2026-01-01
9	9	Deepak Nair	deepak.nair@example.com	Kochi	active	2026-01-01
10	10	Deepak Nair	deepak.nair@example.com	Kochi	active	2026-01-01
11	2	Anjali Verma	anjali.verma@example.com	Delhi	active	2026-01-01
12	11	Sanjay Patel	[null]	Ahmedabad	active	2026-01-01

The table has 12 rows and 7 columns. The status of all rows is 'active'. The updated_at column shows the date '2026-01-01' for all rows. The email column has a null value for the row with customer_id 11.

Output - Raw data saved and displayed as a Delta Table (delta_table)

Step 2 — Data Cleaning

NULL values were removed from the Delta table using `dropna()`, and duplicate rows were removed using `dropDuplicates()`, to maintain good data quality. After cleaning, the final cleaned data was saved back to the same Delta table in overwrite mode.

The screenshot shows a Databricks notebook with several cells. The first cell contains code to read a CSV file into a DataFrame. The second cell contains code to drop NULL values from the DataFrame. The third cell contains code to drop duplicate rows from the DataFrame. The fourth cell contains code to save the cleaned DataFrame back to the Delta table in overwrite mode. The fifth cell contains code to display the cleaned Delta table as a Spark table.

```

# Read CSV file into DataFrame
df_csv = spark.read.csv(
  "/mnt/delta-lake/customers_incremental.csv",
  header=True,
  inferSchema=True
)

# Drop NULL values
df_cleaned = df_csv.dropna()

# Drop duplicate rows
df_cleaned = df_cleaned.dropDuplicates()

# Save cleaned data back to Delta table
df_cleaned.write
  .format("delta")
  .mode("overwrite")
  .saveAsTable("delta_table")

display(spark.table("delta_table"))

```

The output of the fifth cell is a table with 10 rows and 7 columns. The columns are: customer_id, name, email, city, status, and updated_at. The data is as follows:

	customer_id	name	email	city	status	updated_at
1	1	Ravi Sharma	ravi.sharma@example.com	Jaipur	active	2026-01-01
2	2	Anjali Verma	anjali.verma@example.com	Delhi	active	2026-01-01
3	3	Suresh Kumar	suresh.kumar@example.c...	Mumbai	active	2026-01-01
4	4	Priva Singh	priva.singh@example.com	Bangalore	active	2026-01-01
5	5	Amit Joshi	amit.joshi@example.com	Pune	active	2026-01-01
6	6	Neha Gupta	neha.gupta@example.com	Delhi	active	2026-01-01
7	7	Rahul Mehta	rahul.mehta@example.com	Chennai	active	2026-01-01
8	8	Kiran Rao	kiran.rao@example.com	Hyderabad	active	2026-01-01
9	9	Deepak Nair	deepak.nair@example.com	Kochi	active	2026-01-01
10	10	Deepak Nair	deepak.nair@example.com	Kochi	active	2026-01-01

The table has 10 rows and 7 columns. The status of all rows is 'active'. The updated_at column shows the date '2026-01-01' for all rows. The email column has a null value for the row with customer_id 11.

Output - Steps to remove NULL values and duplicates, and save cleaned data back to the Delta table

Step 3 — Incremental Data Load and MERGE (SCD Logic)

A second file, `customer_incremental.csv`, was read, containing new or updated customer records (e.g. changes in city/status). A reference to the target table was then obtained using Delta Lake's `DeltaTable.forName()`, and a MERGE operation was applied:

- whenMatchedUpdateAll() — if a customer_id already exists, that row gets updated (matching records are overwritten).
- whenNotMatchedInsertAll() — if a customer_id is new, that row gets inserted.

This is the core idea behind SCD (Slowly Changing Dimension) — existing records are updated and new entries are added, all in a single atomic operation, without manual delete-and-insert steps.

```
#STEP 4 MERGE OPERATION
from delta.tables import DeltaTable
delta_table = DeltaTable.forName(spark, "delta_table")

(
    delta_table.alias("target")
    .merge(
        df_incremental.alias("source"),
        "target.customer_id = source.customer_id"
    )
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
).show()
```

num_affected_rows	num_updated_rows	num_deleted_rows	num_inserted_rows
6	4	0	2

Output - MERGE operation code and its result — matched rows updated and new rows inserted

Step 4 — Validation

After the MERGE, the final row count of the table was checked, and a duplicate check was performed by grouping on customer_id to confirm the merge worked correctly and no duplicate customer_ids remained.

```
#DUPLICATES CHECK AFTER CLEANING
from pyspark.sql.functions import count

dup_df = spark.table("delta_table").groupBy("customer_id").count().filter("count > 1")
dup_count = dup_df.count()

if dup_count == 0:
    print("No duplicate customer_id values found.")
else:
    print(f"{dup_count} duplicate customer_id value(s) found.")
dup_df.show()
```

dup_df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, count: long]
No duplicate customer_id values found.

Output - Duplicate check code — 'No duplicate customer_id values found' confirms the merge is clean

Step 5 — Final Output

Finally, the updated Delta table was displayed, showing existing records with their updated values (e.g. new city/status/date) as well as newly inserted customers now present in the table.

The screenshot shows a Databricks notebook interface. At the top, the code cell contains the following Python code:

```
#STEP 6: DISPLAY FINAL DATASET
display(spark.table("delta_table"))
```

Below the code, the table view displays 12 rows of data. The table has the following columns: customer_id, name, email, city, status, and updated_at. The data is as follows:

	customer_id	name	email	city	status	updated_at
1	2	Anjali Verma	anjali.verma@example.com	Delhi	active	2026-01-01
2	7	Rahul Mehta	rahul.mehta@example.com	Chennai	active	2026-01-01
3	10	Deepak Nair	deepak.nair@example.com	Kochi	active	2026-01-01
4	4	Priya Singh	priya.singh@example.com	Bangalore	active	2026-01-01
5	5	Amit Joshi	amit.joshi@example.com	Pune	active	2026-01-01
6	9	Deepak Nair	deepak.nair@example.com	Kochi	active	2026-01-01
7	1	Ravi Sharma	ravi.sharma@example.com	Udaipur	active	2026-02-01
8	3	Suresh Kumar	suresh.kumar@example.com	Mumbai	inactive	2026-02-01
9	6	Neha Gupta	neha.gupta.new@example.co...	Delhi	active	2026-02-02
10	11	Meera Iyer	meera.iyer@example.com	Chennai	active	2026-02-02
11	12	Arjun Nair	arjun.nair@example.com	Kochi	active	2026-02-03
12	8	Kiran Rao	kiran.rao@example.com	Hyderabad	active	2026-02-03

At the bottom of the table view, it indicates "12 rows | 1.217s runtime" and "Refreshed 30 minutes ago".

Output - Final Delta table with updated and newly inserted customer records

Summary

- Load: Raw customer data was loaded from CSV and saved into a Delta table.
- Clean: NULL values and duplicate rows were removed.
- Incremental Data: A second CSV was used to simulate new/updated customer records.
- Merge: Delta Lake's MERGE operation updated matching customers and inserted new ones.
- Validate: Row count and duplicate checks confirmed the merge worked correctly.
- Final Output: The updated table was displayed to verify the result.

Conclusion

Delta Lake's MERGE operation enables reliable incremental updates — updating existing records and inserting new ones in a single atomic operation — making it a very useful approach for real-world SCD (Slowly Changing Dimension) use cases.